

Búsqueda informada

Heurísticas, búsqueda voraz y A*

Joaquín F. Sánchez

Curso de Inteligencia Artificial

2026

La información del problema cambia la búsqueda

Búsqueda no informada

Explora con la estructura del espacio de estados, sin estimar qué tan prometedor es cada estado.

Búsqueda informada

Usa conocimiento del dominio para priorizar los estados que parecen conducir con mayor rapidez o menor costo a la meta.

La clave es una función heurística: $h(n)$.

Al finalizar la sesión, el estudiante podrá:

- interpretar una heurística como una estimación del costo restante;
- ejecutar paso a paso la búsqueda voraz y el algoritmo A*;
- comparar $h(n)$, $g(n)$ y $f(n)$;
- evaluar admisibilidad y consistencia de una heurística;
- seleccionar el algoritmo apropiado según calidad, tiempo y memoria.

Una heurística es una estimación útil, no una certeza

Definición

$h(n)$ estima el costo de la mejor ruta desde el estado n hasta una meta.

$$h(n) \approx h^*(n)$$

donde $h^*(n)$ es el costo real óptimo restante, normalmente desconocido.

- Se construye con conocimiento del dominio.
- Debe ser rápida de calcular.
- Cuanto mejor orienta, menos estados innecesarios se expanden.

Ejemplos de heurísticas según el problema

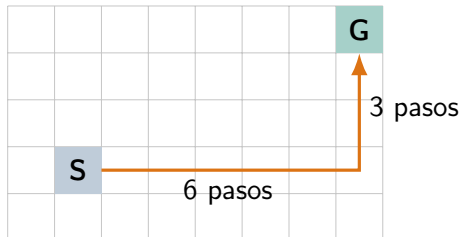
Problema	Posible estimación $h(n)$
Ruta en una ciudad	Distancia en línea recta o tiempo estimado hasta el destino.
Robot en cuadrícula	Distancia Manhattan si solo se mueve horizontal y verticalmente.
Rompecabezas 8-puzzle	Número de fichas fuera de lugar o suma de distancias Manhattan.
Asignación de recursos	Costo mínimo estimado de tareas pendientes y desplazamientos.

Una heurística debe respetar las restricciones reales del problema.

Distancia Manhattan en una cuadrícula

Si $n = (x_n, y_n)$ y la meta es $m = (x_m, y_m)$:

$$h_M(n) = |x_n - x_m| + |y_n - y_m|$$



En este estado, $h_M = 6 + 3 = 9$.

¿Qué hace buena a una heurística?

- **Informativa:** distingue estados prometedores de estados poco útiles.
- **Eficiente:** calcularla cuesta menos que explorar sin orientación.
- **Relevante:** refleja movimientos, obstáculos y costos del dominio.
- **Segura para A*:** idealmente es admisible y consistente.

Equilibrio

Una estimación más precisa puede reducir expansiones, pero puede ser más costosa de calcular.

La búsqueda voraz elige lo que parece más cercano

La búsqueda voraz primero el mejor (Greedy Best-First Search) ordena la frontera usando únicamente:

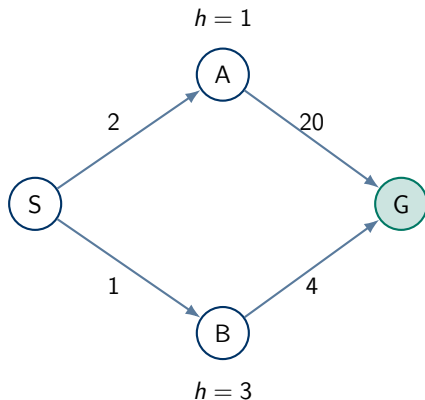
$$f(n) = h(n)$$

Idea central

Expandir el nodo con menor costo estimado hasta la meta, sin considerar cuánto costó llegar hasta él.

Es rápida cuando la heurística guía bien, pero puede escoger rutas caras o callejones sin salida.

Ejemplo: la decisión local puede engañar



- Voraz elige A porque $h(A) = 1 < h(B) = 3$.
- Ruta obtenida: S-A-G, costo 22.
- La ruta S-B-G cuesta solo 5.

Pseudocódigo de búsqueda voraz

Frontera: cola de prioridad por $h(n)$

```
VORAZ(inicio, meta):  
  frontera <- cola_prioridad((h(inicio), inicio))  
  visitados <- conjunto vacío  
  mientras frontera no esté vacía:  
    actual <- extraer_menor_h(frontera)  
    si actual es meta: retornar camino(actual)  
    si actual no está en visitados:  
      agregar actual a visitados  
      para cada vecino de actual:  
        insertar vecino con prioridad  $h(\text{vecino})$   
  retornar fracaso
```

A* equilibra lo recorrido y lo que falta

$$f(n) = g(n) + h(n)$$

$g(n)$

Costo real acumulado desde el inicio hasta n .

$h(n)$

Costo estimado desde n hasta la meta.

$f(n)$

Costo total estimado de una solución que pasa por n .

A* corrige la decisión del ejemplo anterior

Nodo	$g(n)$	$h(n)$	$f(n)$	Prioridad
A	2	1	3	primera
B	1	3	4	segunda
G por A	22	0	22	queda en frontera
G por B	5	0	5	solución

A* no acepta inmediatamente la ruta costosa: conserva alternativas y extrae siempre el menor f .

Estructuras necesarias para implementar A*

- **Frontera abierta:** cola de prioridad ordenada por $f(n)$.
- **Mejor costo conocido:** tabla $g_score[n]$.
- **Predecesores:** tabla $came_from[n]$ para reconstruir la ruta.
- **Conjunto cerrado:** opcional según la implementación y la consistencia de h .

Actualización importante

Si se descubre una ruta más barata hacia un estado, se actualizan su costo y su prioridad.

Pseudocódigo esencial de A*

```
A_ESTRELLA(inicio, meta):  
  abierta <- cola_prioridad((h(inicio), inicio))  
  g[inicio] <- 0  
  padre <- mapa vacío  
  
  mientras abierta no esté vacía:  
    actual <- extraer_menor_f(abierta)  
    si actual es meta: retornar reconstruir(padre, actual)  
  
    para cada vecino de actual:  
      tentativo <- g[actual] + costo(actual, vecino)  
      si tentativo < g.get(vecino, infinito):  
        padre[vecino] <- actual  
        g[vecino] <- tentativo  
        f <- tentativo + h(vecino)  
        insertar_o_actualizar(abierta, vecino, f)  
  retornar fracaso
```

Ejecución manual: registre la frontera en cada paso

- ➊ Inserte el estado inicial con $f = h(\text{inicio})$.
- ➋ Extraiga el nodo con menor f .
- ➌ Calcule para cada sucesor: $g' = g + \text{costo}$ y $f' = g' + h$.
- ➍ Actualice un sucesor solo si el nuevo g' mejora su costo conocido.
- ➎ Repita hasta extraer la meta de la cola de prioridad.
- ➏ Reconstruya el camino siguiendo los predecesores.

Admisibilidad: nunca sobreestimar

Una heurística es **admisibile** si:

$$0 \leq h(n) \leq h^*(n) \quad \text{para todo } n$$

- Es optimista: puede subestimar, pero no excede el costo óptimo restante.
- Debe cumplir $h(meta) = 0$.
- En búsqueda en árbol, A* con heurística admisible encuentra una solución óptima, bajo los supuestos usuales de costos positivos.

Consistencia: respetar el costo de cada transición

Una heurística es **consistente** si para toda arista $n \rightarrow n'$:

$$h(n) \leq c(n, n') + h(n')$$

Es una forma de desigualdad triangular.

Consecuencia

Los valores de f no disminuyen a lo largo de un camino. En búsqueda en grafo, A^* puede cerrar un nodo sin necesitar reabrirlo.

Toda heurística consistente es admisible si $h(meta) = 0$.

Heurística nula: A^* se convierte en costo uniforme

Si para todo estado:

$$h(n) = 0,$$

entonces:

$$f(n) = g(n).$$

A^* se comporta como búsqueda de costo uniforme: mantiene optimalidad, pero pierde la orientación hacia la meta.

Interpretación

La heurística no cambia qué significa una solución óptima; cambia cuánto trabajo se necesita para encontrarla.

Dominar significa informar mejor sin sobreestimar

Sean h_1 y h_2 heurísticas admisibles. Si

$$h_2(n) \geq h_1(n) \quad \text{para todo } n,$$

entonces h_2 **domina** a h_1 .

- A^* con h_2 suele expandir igual o menos nodos.
- En el 8-puzzle, Manhattan domina al número de fichas fuera de lugar.
- La comparación solo es útil si el costo de calcular h_2 es razonable.

Voraz prioriza rapidez; A* incorpora calidad de ruta

	Voraz	A*
Prioridad	$h(n)$	$g(n) + h(n)$
Costo recorrido	Lo ignora	Lo considera
Optimalidad	No garantizada	Sí, con condiciones sobre h
Completitud	Depende del espacio y control de repetidos	Sí, con ramificación finita y costos positivos
Memoria	Puede ser alta	Generalmente alta
Uso típico	Respuesta rápida aproximada	Ruta de costo mínimo

Complejidad: la calidad de h es decisiva

- En el peor caso, búsqueda voraz y A^* pueden expandir un número exponencial de nodos.
- A^* conserva en memoria la frontera y los costos conocidos; la memoria suele ser su principal limitación.
- Una buena heurística reduce el número efectivo de estados explorados.
- Una heurística demasiado costosa puede anular el ahorro obtenido.

Pregunta práctica

¿Conviene invertir más tiempo en calcular $h(n)$ si evita miles de expansiones?

Errores frecuentes al implementar A^*

- Usar una pila o cola FIFO en lugar de una cola de prioridad.
- Marcar un estado como visitado sin comparar costos alternativos.
- Detenerse al *generar* la meta, en vez de hacerlo cuando se extrae con mínima prioridad.
- Confundir $h(n)$ con el costo acumulado $g(n)$.
- Emplear una heurística que sobreestima y asumir que la ruta será óptima.
- No definir un criterio estable para empates.

Actividad guiada: diseñar una heurística

Un robot debe transportar materiales en una bodega con pasillos, obstáculos y zonas de distinta velocidad.

- 1 Defina el estado, las acciones, la meta y el costo de cada acción.
- 2 Proponga dos heurísticas distintas.
- 3 Analice si cada heurística es admisible.
- 4 Prediga cuál expandirá menos nodos y justifique.
- 5 Compare el resultado esperado de voraz y A^* .

Ejercicio de cierre

Considere los nodos de la frontera:

Nodo	$g(n)$	$h(n)$	$f(n)$
P	7	2	9
Q	3	5	8
R	1	6	7

- 1 ¿Qué nodo selecciona la búsqueda voraz?
- 2 ¿Qué nodo selecciona A*?
- 3 ¿Por qué las decisiones son diferentes?

Ejercicio de cierre

Considere los nodos de la frontera:

Nodo	$g(n)$	$h(n)$	$f(n)$
P	7	2	9
Q	3	5	8
R	1	6	7

- 1 ¿Qué nodo selecciona la búsqueda voraz?
- 2 ¿Qué nodo selecciona A*?
- 3 ¿Por qué las decisiones son diferentes?

Respuesta: voraz selecciona P; A* selecciona R.

Síntesis: elegir bien la función de evaluación

- $h(n)$ incorpora conocimiento del dominio.
- Voraz usa $h(n)$: puede avanzar rápido, pero no asegura el menor costo.
- A* usa $g(n) + h(n)$: equilibra evidencia acumulada y estimación futura.
- Admisibilidad y consistencia permiten razonar sobre optimalidad y eficiencia.
- La implementación correcta requiere cola de prioridad y actualización de costos.

Una buena heurística guía; una buena evaluación decide.

-  Material suministrado para la sesión. *Intelligent Search*, capítulo 3, pp. 59–70.
-  Russell, S. y Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson.
-  Hart, P.; Nilsson, N.; Raphael, B. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". *IEEE Transactions on Systems Science and Cybernetics*, 1968.